



Esercizio 3: Navigare nel Filesystem Linux e Impostazioni dei Permessi

In questo laboratorio, prenderai familiarità con i filesystem Linux.

- Parte 1- Esplorare i **Filesystem** in Linux
- Parte 2- **Permessi** dei File
- Parte 3- Link **Simbolici** e Altri Tipi di File Speciali



Parte 1: Esplorare i FileSystem

Questo laboratorio si concentra sulla famiglia **ext**, uno dei filesystem più comuni trovati su Linux.

Passo 1 - Accedere alla riga di comando. Avvia la VM **CyberOps** Workstation e apri una finestra di terminale.

Passo 2- Visualizzare i **filesystem** attualmente montati. I filesystem devono essere montati prima di poter essere accessibili e utilizzati. In informatica, montare un filesystem significa renderlo accessibile al sistema operativo. Montare un filesystem è il processo di collegare la partizione fisica sul dispositivo a **blocchi** (hard disk, unità SSD, pen drive, ecc.) a una directory, attraverso la quale è possibile accedere all'intero filesystem. Poiché la suddetta directory diventa la **radice** del filesystem appena montato, è anche conosciuta come punto di montaggio (mounting point). a. Usa il comando **lsblk** per visualizzare tutti i dispositivi a blocchi:

```
[analyst@secOps ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda          8:0    0   10G  0 disk
└─sda1       8:1    0   10G  0 part /
sdb          8:16   0    1G  0 disk
└─sdb1       8:17   0 1023M  0 part
sr0         11:0    1 1024M  0 rom
```

L'output sopra mostra che la VM CyberOps Workstation ha tre dispositivi a blocchi installati: **sr0**, **sda** e **sdb**. L'output ad albero mostra anche partizioni sotto sda e sdb.

Convenzionalmente, **/dev/sdX** è usato da Linux per rappresentare gli hard disk, con il numero finale che rappresenta il numero della **partizione** all'interno di quel dispositivo.

Computer con più hard disk mostrerebbero probabilmente più dispositivi **/dev/sdX**. Se Linux fosse in esecuzione su un computer con quattro hard disk, ad esempio, li mostrerebbe come **/dev/sda**, **/dev/sdb**, **/dev/sdc** e **/dev/sdd**, per impostazione predefinita. L'output implica che sda e sdb sono hard disk, ognuno contenente una singola partizione. L'output mostra anche che sda è un disco da 10GB mentre sdb ha 1GB.

Nota: Linux spesso visualizza anche le chiavette USB come **/dev/sdX**, a seconda del tipo di firmware.

b. Usa il comando **mount** per visualizzare informazioni più **dettagliate** sui filesystem attualmente montati nella VM CyberOps Workstation.

```
[analyst@secOps ~]$ mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sys on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
dev on /dev type devtmpfs (rw,nosuid,relatime,size=1038408k,nr_inodes=218258,mode=755)
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755)
/dev/sda1 on / type ext4 (rw,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=880)
tmpfs on /sys/fs/cgroup type tmpfs (ro,nosuid,nodev,noexec,mode=755)
<output omissa>
```

Molti dei filesystem sopra sono fuori dallo scopo di questo corso e irrilevanti per il laboratorio. Concentriamoci sul **filesystem radice (root filesystem)**, il filesystem memorizzato in **/dev/sda1**.

Il filesystem radice è dove è memorizzato il sistema operativo Linux stesso; tutti i programmi, gli strumenti, i file di configurazione sono memorizzati nel filesystem radice per impostazione **predefinita**.

c. Esegui di nuovo il comando `mount`, ma questa volta, usa la pipe `|` per inviare l'output di **mount** a **grep** per filtrare l'output e visualizzare solo il filesystem radice:

```
[analyst@secOps ~]$ mount | grep sda1
/dev/sda1 on / type ext4 (rw,relatime)
```

L'output ci dice anche il tipo di **formattazione** usato nella partizione, **ext4** in questo caso. Le informazioni tra parentesi si riferiscono alle opzioni di montaggio della partizione. d. Esegui i seguenti due comandi sulla VM CyberOps Workstation: **ls -l**

```
[analyst@secOps ~]$ cd /
[analyst@secOps /]$ ls -l
total 52
lrwxrwxrwx  1 root root    7 May  3 15:26 bin -> usr/bin
drwxr-xr-x  3 root root 4096 Jun 18 19:07 boot
drwxr-xr-x 20 root root 3920 Sep 29 07:45 dev
drwxr-xr-x 73 root root 4096 Jun 19 04:45 etc
drwxr-xr-x  3 root root 4096 Mar 20  2018 home
lrwxrwxrwx  1 root root    7 May  3 15:26 lib -> usr/lib
lrwxrwxrwx  1 root root    7 May  3 15:26 lib64 -> usr/lib64
drwx----- 2 root root 16384 Mar 20  2018 lost+found
drwxr-xr-x  2 root root 4096 Jan  5  2018 mnt
drwxr-xr-x  3 root root 4096 Jun 17 15:07 opt
dr-xr-xr-x 204 root root    0 Sep 29 07:45 proc
drwxr-xr-x  8 root root 4096 Jun 18 20:09 root
drwxr-xr-x 22 root root  600 Sep 29 07:54 run
lrwxrwxrwx  1 root root    7 May  3 15:26/sbin -> usr/sbin
drwxr-xr-x  6 root root 4096 Mar 24  2018 srv
dr-xr-xr-x 13 root root    0 Sep 29 07:45 sys
```



Qual è il significato dell'output? Dove sono fisicamente memorizzati i file elencati? Perché /dev/sdb1 non viene mostrato nell'output sopra?

Il significato dell'output è: **cd /** mi sposta nella directory di root, mentre **ls** mostra file e sotto-directory presenti nella directory corrente (quindi quella di root), e **-l** visualizza ulteriori dettagli, come ad esempio i permessi dei vari file/cartelle. Basandosi sui passaggi precedenti, possiamo dire che i **file/cartelle** sono salvati fisicamente nella partizione **sda1** della memoria **sda**. Il motivo per cui non si vede **sdb1** è perché l'output di **mount** è stato filtrato con il **grep** per vedere solamente **sda1**.

Passo 3- Montare e smontare manualmente i filesystem

Il comando **mount** può anche essere usato per montare e smontare i filesystem. Come visto nel Passo 1, la VM CyberOps Workstation ha due hard disk installati. Il primo è stato riconosciuto dal kernel come `/dev/sda` mentre il secondo è stato riconosciuto come `/dev/sdb`. Prima che un dispositivo a blocchi possa essere montato, deve avere un punto di **montaggio**.

a. Usa il comando **ls -l** per verificare che la directory `second_drive` sia nella directory home dell'utente `analyst`.

```
[analyst@secOps ~]$ ls -l second_drive/  
total 0
```

Nota che la directory è **vuota**. c.

Usa il comando `mount` per montare `/dev/sdb1` sulla directory `second_drive` appena creata. La sintassi di `mount` è: **mount [opzioni] <dispositivo_da_montare> <punto_di_montaggio>**.

```
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/  
[sudo] password for analyst:
```

Nessun output viene fornito, il che significa che il processo di montaggio è stato completato con **successo**.

d. Ora che `/dev/sdb1` è stato montato su `/home/analyst/second_drive`, usa **ls -l** per elencare di nuovo il contenuto della directory.

```
[analyst@secOps ~]$ ls -l second_drive/  
total 20  
drwx----- 2 root    root    16384 Mar  3 10:59 lost+found  
-rw-r--r--  1 root    root      183 Mar  3 15:42 myFile.txt
```



Perché la directory non è più vuota? Dove sono fisicamente memorizzati i file elencati?

I dati sono memorizzati in **sdb1** e sono accessibili tramite il **mount** sulla cartella `second_drive`, che non è più vuota.

e. Esegui di nuovo il comando `mount` senza opzioni per visualizzare informazioni dettagliate sulla partizione `/dev/sdb1`. Come prima, usa il comando `grep` per visualizzare solo i filesystem `/dev/sdX`:

```
[analyst@secOps ~]$ mount | grep /dev/sd
/dev/sda1 on / type ext4 (rw,relatime)
/dev/sdb1 on /home/analyst/second_drive type ext4 (rw,relatime)
```

f. Smontare i filesystem è altrettanto semplice. Assicurati di cambiare la directory in qualcosa al di fuori del punto di montaggio e usa il comando `umount`, come mostrato sotto:

```
[analyst@secOps ~]$ sudo umount /dev/sdb1
[sudo] password for analyst:
[analyst@secOps ~]$
[analyst@secOps ~]$ ls -l second_drive/
total 0
```

Parte 2: Permessi dei File

I filesystem Linux hanno funzionalità integrate per controllare la capacità degli utenti di visualizzare, modificare, navigare ed eseguire i contenuti del filesystem. Essenzialmente, ogni file nei filesystem porta con sé il proprio set di permessi, definendo sempre cosa utenti e gruppi possono fare con il file.

Passo 1- Visualizzare e Modificare i Permessi dei File.

a. Naviga in /home/analyst/lab.support.files/scripts/.

```
[analyst@secOps ~]$ cd lab.support.files/scripts/
```

b. Usa il comando ls -l per visualizzare i permessi dei file.

```
[analyst@secOps scripts]$ ls -l
total 60
-rwxr-xr-x 1 analyst analyst 190 Jun 13 09:45 configure_as_dhcp.sh
-rwxr-xr-x 1 analyst analyst 192 Jun 13 09:45 configure_as_static.sh
-rwxr-xr-x 1 analyst analyst 3459 Jul 18 10:09 cyberops_extended_topo_no_fw.py
-rwxr-xr-x 1 analyst analyst 4062 Jul 18 10:09 cyberops_extended_topo.py
-rwxr-xr-x 1 analyst analyst 3669 Jul 18 10:10 cyberops_topo.py
-rw-r--r-- 1 analyst analyst 2871 Apr 28 11:27 cyops.mn
-rwxr-xr-x 1 analyst analyst 458 May 1 13:50 fw_rules
-rwxr-xr-x 1 analyst analyst 70 Apr 28 11:27 mal_server_start.sh
drwxr-xr-x 2 analyst analyst 4096 Jun 13 09:55 net_configuration_files
-rwxr-xr-x 1 analyst analyst 65 Apr 28 11:27 reg_server_start.sh
-rwxr-xr-x 1 analyst analyst 189 Dec 15 2016 start_ELK.sh
-rwxr-xr-x 1 analyst analyst 85 Dec 22 2016 start_miniedit.sh
```



Considera il file cyops.mn come esempio. Chi è il proprietario del file? E il gruppo? I permessi per cyops.mn sono -rw-r-r-. Cosa significa?

Il proprietario del file è l'utente analyst, che appartiene al gruppo analyst. -rw-r-r- significa che: il proprietario ha permessi di scrittura e lettura del file, mentre il gruppo di cui fa parte ha permessi di sola lettura e gli altri utenti hanno a loro volta permessi di sola lettura.



Il comando **touch** è molto semplice e utile. Permette la creazione rapida di un file di testo vuoto. Usa il comando sottostante per creare un file vuoto nella directory /mnt:

```
[analyst@secOps scripts]$ touch /mnt/myNewFile.txt  
touch: cannot touch '/mnt/myNewFile.txt': Permission denied
```



Perché il file non è stato creato? Elenca i permessi, la proprietà e il contenuto della directory /mnt e spiega cosa è successo. Con l'aggiunta dell'opzione -d, elenca i permessi della directory genitore. Registra la risposta nelle righe sottostanti.

```
[analyst@secOps ~]$ ls -ld /mnt  
drwxr-xr-x 2 root root 4096 Jan 5 2018 /mnt
```

Cosa si può fare affinché il comando touch mostrato sopra abbia successo?

```
[analyst@secOps scripts]$ sudo touch /mnt/MyNewFile.txt  
[sudo] password for analyst:  
[analyst@secOps scripts]$ cd /mnt  
[analyst@secOps mnt]$ ls  
MyNewFile.txt
```

Il file non è stato creato perché **non abbiamo** i permessi con l'utente analyst per poter scrivere nella cartella. Lanciano il comando `ls -ld /mnt` possiamo vedere quali permessi a quali utenti sono assegnati alla cartella. Il proprietario, l'utente root, ha permessi di **lettura scrittura** ed esecuzione, gli utenti del gruppo root permessi di lettura ed esecuzione, mentre gli altri utenti solo permessi di esecuzione. Siccome l'utente root è l'unico con i **permessi** necessari, per far sì che il comando touch abbia successo basta aggiungere sudo all'inizio del comando. In alternativa, si potrebbe usare il comando sudo **chmod** per allargare i permessi di scrittura anche ad altri utenti.

d. Il comando `chmod` è usato per cambiare i permessi di un file o di una directory. Come prima, monta la partizione **/dev/sdb1** sulla directory **/home/analyst/second_drive** creata precedentemente in questo laboratorio:

```
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/
```

e. Cambia nella directory `second_drive` ed elenca il suo contenuto:

```
[analyst@secOps ~]$ cd ~/second_drive
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root    root    16384 Mar  3 10:59 lost+found
-rw-r--r--  1 root    root      183 Mar  3 15:42 myFile.txt
```

f. Usa il comando `chmod` per cambiare i permessi di **myFile.txt**.

```
[analyst@secOps second_drive]$ sudo chmod 665 myFile.txt
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root    root    16384 Mar  3 10:59 lost+found
-rw-rw-r-x  1 root    root      183 Mar  3 15:42 myFile.txt
```



Quali sono i permessi del file `myFile.txt`? Sono cambiati?

I permessi iniziali dei file sono: **di lettura e scrittura** per il proprietario, ovvero l'utente `root`; di sola lettura per gli altri utenti. Con `chmod`, si sono modificati i permessi come segue: gli utenti del gruppo `root` a cui appartiene il proprietario del file hanno ottenuto permessi di scrittura oltre che di lettura, mentre tutti gli altri utenti hanno ottenuto i permessi di **esecuzione**.

Il comando **chmod** prende i permessi nel formato ottale. In questo modo, una scomposizione di 665 è la seguente:

6 in ottale è 110 in binario. Assumendo che ogni posizione dei permessi di un file possa essere 1 o 0, 110 significa rw- (lettura=1, scrittura=1 ed esecuzione=0).

Pertanto, il comando **chmod 665 myFile.txt** cambia i permessi a:

Proprietario: rw- 6 in ottale o 110 in binario)

Gruppo: rw- 6 in ottale o 110 in binario)

Altri: r-x 5 in ottale o 101 in binario)

drwxrwxrwx

d = Directory
r = Read
w = Write
x = Execute

chmod 777

rwX|rwX|rwX
Owner|Group|Others

7	rwX	111
6	rw-	110
5	r-X	101
4	r--	100
3	-wX	011
2	-w-	010
1	--X	001
0	---	000



Quale comando cambierebbe i permessi di **myFile.txt** a **rwXrwxrwx**, garantendo a qualsiasi utente nel sistema pieno accesso al file?

Il comando è **chmod 777 myFile.txt**

g. Il comando **chown** è usato per cambiare la proprietà di un file o di una directory. Esegui il comando sottostante per rendere root il proprietario di **myFile.txt**:

```
[analyst@sec0ps second_drive]$ sudo chown analyst myFile.txt
[sudo] password for analyst:
[analyst@sec0ps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar 3 10:59 lost+found
-rw-rw-r-x 1 analyst root 183 Mar 3 15:42 myFile.txt
[analyst@sec0ps second_drive]$
```

Nota: Per cambiare sia il proprietario che il gruppo in **analyst** allo stesso tempo, usa il formato **sudo chown analyst:analyst myFile.txt**.

h. Ora che **analyst** è il proprietario del file, prova ad accodare la parola 'test' alla fine di **myFile.txt**.

```
[analyst@sec0ps second_drive]$ echo test >> myFile.txt
[sudo] password for analyst:
[analyst@sec0ps second_drive]$ cat myFile.txt
```



L'operazione è riuscita?

Grazie al cambio di proprietario per il file nel nostro utente analyst, otteniamo di poter scrivere all'interno del file myFile.txt. Si veda lo screen seguente a titolo dimostrativo.

```
[analyst@secOps ~]$ ls -l ~/second_drive
total 20
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-r--r-- 1 analyst analyst  183 Mar 26  2018 myFile.txt
[analyst@secOps ~]$ echo test >> myFile.txt
[analyst@secOps ~]$ cat myFile.txt
test
[analyst@secOps ~]$
```

Passo 2: Directory e Permessi

Simile ai file regolari, anche le directory portano permessi. Sia i file che le directory hanno 9 bit per i permessi del proprietario/utente, del gruppo e degli altri. Ci sono anche altri tre bit per permessi speciali: setuid, setgid e sticky bit, che vanno oltre lo scopo di questo laboratorio.

a. Torna alla directory /home/analyst/lab.support.files ed esegui il comando `ls -l` per elencare tutti i file con dettagli:

```
[analyst@secOps second_drive]$ cd ~/lab.support.files/
[analyst@secOps lab.support.files]$ ls -l
total 580
-rw-r--r-- 1 analyst analyst 649 Jun 28 18:34 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Jun 28 11:13 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Aug 7 15:29 attack_scripts
-rw-r--r-- 1 analyst analyst 182 Jul 20 09:37 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Dec 15 2016 cyops.mn
-rw-r--r-- 1 analyst analyst 75 May 24 11:07 elk_services
-rw-r--r-- 1 analyst analyst 373 Feb 16 16:04 h2_dropbear.banner
-rw-r--r-- 1 analyst analyst 147 Mar 21 15:30 index.html
-rw-r--r-- 1 analyst analyst 255 May 2 13:11 letter_to_grandma.txt
-rw-r--r-- 1 analyst analyst 24464 Feb 7 2017 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst 4096 May 25 13:01 malware
-rwxr-xr-x 1 analyst analyst 172 Jul 25 16:27 mininet_services
drwxr-xr-x 2 analyst analyst 4096 Feb 14 2017 openssl_lab
drwxr-xr-x 2 analyst analyst 4096 Aug 7 15:25 pcaps
drwxr-xr-x 7 analyst analyst 4096 Sep 20 2016 pox
-rw-r--r-- 1 analyst analyst 473363 Feb 16 15:32 sample.img
-rw-r--r-- 1 analyst analyst 65 Feb 16 15:45 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst 4096 Jul 18 18:18 scripts
-rw-r--r-- 1 analyst analyst 25553 Feb 13 2017 SQL_Lab.pcap
```

Confronta i permessi della directory **malware** con il file **mininet_services**.

Qual è la differenza tra la parte iniziale della riga di malware e la riga di mininet_services?

La lettera 'd' all'inizio della riga indica che il tipo di file è una directory e non un file. Un'altra differenza tra i permessi di file e directory è il bit di esecuzione. Se un file ha il suo bit di esecuzione attivato, significa che può essere eseguito dal sistema. Le directory sono diverse dai file con il bit di esecuzione impostato (un file con il bit di esecuzione impostato è uno script o programma eseguibile). Una directory con il bit di esecuzione impostato specifica se un utente può entrare in quella directory. I comandi `chmod` e `chown` funzionano per le directory nello stesso modo in cui funzionano per i file.



La differenza è che **malware** è una **directory** e quindi ha la dicitura **d**, mentre **mininet_services** è un file ed ha infatti la dicitura **-**.

Parte 3: Link Simbolici e Altri Tipi di File Speciali

Hai visto alcuni dei diversi tipi di file in Linux. Il primo carattere in ogni file elencato con il comando `ls -l` mostra il tipo di file. I tre diversi tipi di file in Linux, inclusi i loro sottotipi e caratteri, sono:

- **File Regolari (-)** che includono:
 - File leggibili - file di testo
 - File binari - programmi
 - File immagine
 - File compressi
- **File Directory (d)**
 - Cartelle
- **File Speciali** che includono:
 - **File a Blocco (b)** - File usati per accedere all'hardware fisico come punti di montaggio per accedere agli hard disk.
 - **File a Carattere (c)** - File che forniscono un flusso seriale di input e output. I terminali tty sono esempi di questo tipo di file.
 - **File Pipe (p)** - Un file usato per passare informazioni dove i primi byte in entrata sono i primi byte in uscita. Questo è anche noto come FIFO (first in first out).
 - **File Link Simbolici (l)** - File usati per collegarsi ad altri file o directory. Ci sono due tipi: link simbolici e hard link.
 - **File Socket (s)** - Questi sono usati per passare informazioni da applicazione ad applicazione per comunicare su una rete.

Passo 1: Esaminare i tipi di file.

a. Usa il comando `ls -l` per visualizzare i file nella cartella `/home/analyst`. Nota che i primi caratteri di ogni riga sono o un `"-"` che indica un file o un `"d"` che indica una directory.

```
[analyst@secOps ~]$ ls -l
total 28
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Mar  3 18:23 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

b. Produci un elenco della directory **/dev**. Scorri fino al centro dell'output e nota come i file a blocco iniziano con una "b", i file a carattere iniziano con una "c" e i file link simbolici iniziano con una "l".

```
[analyst@secOps ~]$ ls -l /dev/
<output omissa>
crw-rw-rw- 1 root tty      5,  2 May 29 18:32 ptmx
drwxr-xr-x 2 root root      0 May 23 06:40 pts
crw-rw-rw- 1 root root      1,  8 May 23 06:41 random
crw-rw-r-- 1 root root    10, 56 May 23 06:41 rfkill
lrwxrwxrwx 1 root root      4 May 23 06:41 rtc -> rtc0
crw-rw---- 1 root audio 253,  0 May 23 06:41 rtc0
brw-rw---- 1 root disk     8,  0 May 23 06:41 sda
brw-rw---- 1 root disk     8,  1 May 23 06:41 sda1
brw-rw---- 1 root disk     8, 16 May 23 06:41 sdb
brw-rw---- 1 root disk     8, 17 May 23 06:41 sdb1
drwxrwxrwt 2 root root     40 May 28 13:47 shm
crw----- 1 root root    10, 231 May 23 06:41 snapshot
drwxr-xr-x 2 root root     80 May 23 06:41 snd
brw-rw----+ 1 root optical 11,  0 May 23 06:41 sr0
lrwxrwxrwx 1 root root     15 May 23 06:40 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root     15 May 23 06:40 stdin -> /proc/self/fd/0
lrwxrwxrwx 1 root root     15 May 23 06:40 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty      5,  0 May 29 17:36 tty
crw--w---- 1 root tty      4,  0 May 23 06:41 tty0
<output omissa>
```

c. I link simbolici in Linux sono come i collegamenti in Windows. Ci sono due tipi di link in Linux: link simbolici e hard link. La differenza tra link simbolici e hard link è che un file link simbolico punta al nome file di un altro file e un file hard link punta al contenuto di un altro file. Crea due file

```
[analyst@secOps ~]$ echo "symbolic" > file1.txt
[analyst@secOps ~]$ cat file1.txt
symbolic
[analyst@secOps ~]$ echo "hard" > file2.txt
[analyst@secOps ~]$ cat file2.txt
hard
```

d. Usa **ln -s** per creare un link simbolico a file1.txt, e **ln** per creare un hard link a file2.txt:

```
[analyst@secOps ~]$ ln -s file1.txt file1symbolic
[analyst@secOps ~]$ ln file2.txt file2hard
```

e. Usa il comando `ls -l` ed esamina l'elenco della directory:

```
[analyst@secOps ~]$ ls -l
total 40
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
lrwxrwxrwx 1 analyst analyst   9 Aug 17 16:43 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst   9 Aug 17 16:41 file1.txt
-rw-r--r-- 2 analyst analyst   5 Aug 17 16:42 file2hard
-rw-r--r-- 2 analyst analyst   5 Aug 17 16:42 file2.txt
drwxr-xr-x 9 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Mar  3 18:23 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

Nota come il file **file1symbolic** sia un link simbolico con una **l** all'inizio della riga e un puntatore $\rightarrow$ a **file1.txt**. Il **file2hard** sembra essere un file regolare, perché di fatto è un file regolare che punta allo stesso inode sull'hard disk di **file2.txt**. In altre parole, **file2hard** punta agli stessi attributi e posizione del blocco disco di **file2.txt**. Il numero 2 nella quinta colonna dell'elenco per **file2hard** e **file2.txt** indica che ci sono 2 file collegati tramite hard link allo stesso inode. Per un elenco di directory, la quinta colonna indica il numero di directory all'interno della directory, incluse le cartelle nascoste.

f. Cambia i nomi dei file originali: **file1.txt** e **file2.txt**, e nota come influisce sui file collegati.

```
[analyst@secOps ~]$ mv file1.txt file1new.txt
[analyst@secOps ~]$ mv file2.txt file2new.txt
[analyst@secOps ~]$ cat file1symbolic
cat: file1symbolic: no such file or directory

[analyst@secOps ~]$ cat file2hard
hard
```

Nota come **file1symbolic** sia ora un link simbolico rotto perché il nome del file a cui puntava, **file1.txt**, è cambiato, ma il file hard link **file2hard** funziona ancora correttamente perché punta all'inode di **file2.txt** e non al suo nome, che ora è **file2new.txt**.



Cosa pensi succederebbe a **file2hard se aprissi un editor di testo e cambiassi il testo in **file2new.txt**?**

Se cambiassi il testo in **file2new.txt**, otterrei la stessa modifica anche in **file2hard.txt**, in quanto entrambi puntano allo stesso inode.